

第六章 Turbo码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

Turbo码的研究背景

- 香农有信道编码定理指出：在信道传输速率 R 不超过信道容量 C 的前提下，只有在码组长度无限的码集合中随机地选择编码码字并且在接收端采用最大似然译码算法时，才能逼近香农信道容量，误码率接近于零。
- 但是传统的编码都有规则的代数结构，远远谈不上“随机”；而且最大似然译码的复杂性随编码长度的增加而加大，当编码长度趋于无穷大时，最大似然译码是不可能实现的。出于译码复杂度的考虑，码长也不可能太长。因此，传统的信道编码性能与信道容量之间都有较大的差距。

Turbo码的研究背景

- 事实上，长期以来信道容量仅作为一个理论极限存在，实际的编码方案设计和评估都没有以香农限为依据。
- 随机性编译码被认为仅仅是为证明定理存在性而引入的一种数学方法和手段，在实际的编码构造中是不可能实现的。
- 1993年，两位任教于法国不列颠通信大学的教授C. Berrou、A. Glavieux和他们的缅甸籍博士生P. Thitimajshima在瑞士日内瓦召开的国际通信会议(ICC)上发表的论文《Near Shannon limit error-correcting coding and decoding: Turbo codes》中，提出了一种新的编码方法，即Turbo码。

Turbo码的研究背景

- C. Berrou等人提出的Turbo码就是并行级联卷积码，巧妙地将卷积码和伪随机交织器结合在一起，在实现随机编码思想的同时，通过交织器实现了由短码构造长码的方法，并在两个软入/软出(SISO)译码器之间进行多次迭代来逼近最大似然译码。
- Turbo码很好地应用了香农信道编码定理中的随机性编、译码条件，从而获得了几乎接近香农理论极限的译码性能。
- 从此以后，Turbo码得到了广泛的关注和发展，并对当今的编码理论和研究方法产生了深远的影响，信道编码理论的研究也随之进入了一个新的阶段。

课程思政

- Turbo码是一种级联码，利用迭代算法，以时间换取复杂度，第一次实现了现实可行的对香农极限的逼近。
- Turbo码的发明让人茅塞顿开，它不再是一种简单的编解码方案，而成为一种思想，人们基于这一思想开发了Turbo均衡、Turbo信道估计等应用，Turbo思想将信息理论推向了一个新的台阶。
- 而Turbo码在3G和4G中的应用，正是移动互联网大爆发的年代，它在无形之中服务信息社会，支撑其对数据传输的需求。
- Turbo码的发明的启示：就像学习中遇到的问题一样，解决方法并不是唯一的，大家要勤思考，找到解决问题的办法，提升自我解决问题的能力

Turbo码的优点

- 在信噪比较低的高噪声环境下性能优越（信道条件差的移动通信系统中有很大的应用潜力），而且具有很强的抗衰落、抗干扰能力。
- Turbo码具有超乎寻常的优异的译码性能，可以纠正高速率数据传输时发生的误码。在直扩（CDMA）系统中采用Turbo码技术可以进一步提高系统的容量。
- 在短帧情况下的仿真结果表明短交织Turbo码在AWGN信道和Rayleigh衰落信道下仍具有接近信道容量的纠错能力。

Turbo码的缺点

- 为获得较高的性能，Turbo码的编译码方式较为复杂。
- 具有较大的译码时延，这是由于block长度较大、译码需要多次迭代造成的。这样非常不利实时业务或高速数据的传输。
- BER(比特出错概率)在 10^{-5} 后会出现误码平台，这是由于Turbo码的重量分布造成的。对于某些对BER要求较高的应用就不适合，当然通过交织器的设计能够提供更大的码间最小距离，从而降低误码平台。

分量码

- 卷积码的类型多样，如非递归卷积码(NRC)、非系统卷积码(NSC)、递归系统卷积码(RSC)。
- 在实际的系统中，由于RSC有突出的优点而被广泛采用。RSC码在具有良好的译码性能，且存在较高的交织增益，译码性能也会随着交织长度增加而提高。
- 影响译码性能的因素很多，如卷积码的约束长度、生成多项式以及生成多项式的顺序等，因此，在实际中用不同种类的分量码构造的Turbo码，需要根据不同的场景和用户的需求，经过仿真和验证才能获得。

交织器

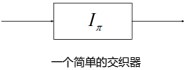
- 根据香农理论，码长越长，编码增益越大。因此，交织器的长度越长，Turbo码的码长越长，性能越好。
- Turbo码在交织器长度很长时可以获得优于卷积码的性能，但是在移动通信中短长度的帧是比较常见的，而在同等复杂度的情况下，Turbo码的性能是不如卷积码的。
- 另外，交织器的交织规则也是影响因素。最简单的交织器即行列交织器，数据按行写入，按列读出。
- 选用交织器时要根据交织器长度、交织规则、系统复杂度等综合考虑。
- 有分组交织器、伪随机交织器、循环移位交织器、分组螺旋交织器、二次算术交织器等。

Turbo码的应用

- 移动通信系统、卫星通信系统、数字音频广播、数字视频广播、深空通信、UMTS/3GPP、CDMA、信息隐藏领域、分布式信源编码、信源信道联合编码。
- 3G移动通信因为其业务类型对信道编码提出了更高的要求，而相应的WCDMA和CDMA2000方案都建议采用除与IS-95 CDMA系统类似的卷积编码技术和交织技术之外，采用Turbo编码技术。
- CDMA2000中，Turbo码被用在CDMA系统前向、反向链路信道中，RSC(递归系统卷积)编码器基于8状态的并行级联卷积码(8PCCC)。
- 在WCDMA中，Turbo码交织器是可载短型块交织器。

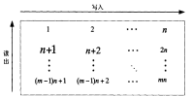
交织器

- 交织器的主要功能是使码重分布合理，降低数据序列的相关性，增大输出码字的最小汉明距，实现随机编码。
- 交织器将原始数据顺序按照给定的规则重新排序输出，能把通信产生的连续错误变成许多离散的单个错误，本质上改变序列的结构而不改变数据序列的内容。解交织即是交织的逆过程。



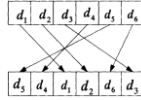
分组交织器

- **分组交织器**：将数据序列按行的顺序写入 $m \times n$ 的矩阵，然后按列的顺序读出，即完成交织过程。相应的解交织过程就是将交织后的数据序列按列写入 $m \times n$ 矩阵，再按照行的顺序读出。



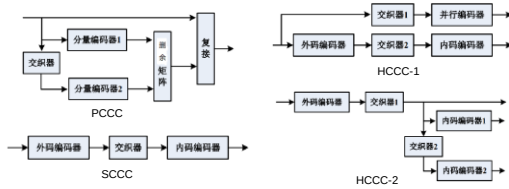
伪随机交织器

- **伪随机交织器**：指交织映射随机生成的交织器，每个长度为 N 的伪随机交织器共有 $N!$ 种可能的交织形式。这种交织器采用给定的随机地址交织映射，由这个已知的交织表对输入信息序列进行映射。



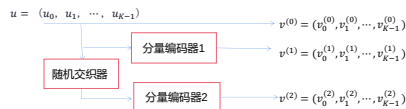
Turbo码编码器的结构

- 三种：并行级联卷积码(PCCC)、串行级联卷积码(SCCC)和混合级联卷积码(HCCC)



并行级联卷积码(PCCC)

- 第一个输出序列 = 信息序列，即 $u = v^{(0)} = (v_0^{(0)}, v_1^{(0)}, \dots, v_{K-1}^{(0)})$
- 第一个编码器输出的校验序列为 $v^{(1)} = (v_0^{(1)}, v_1^{(1)}, \dots, v_{K-1}^{(1)})$
- 交织器对 K 个比特进行排序处理，得到 u' ，第二个编码器输出的校验序列为 $v^{(2)} = (v_0^{(2)}, v_1^{(2)}, \dots, v_{K-1}^{(2)})$
- 最终的发送序列(码字)为 $v = (v_0^{(0)}, v_0^{(1)}, v_0^{(2)}, v_1^{(0)}, v_1^{(1)}, v_1^{(2)}, \dots, v_{K-1}^{(0)}, v_{K-1}^{(1)}, v_{K-1}^{(2)})$

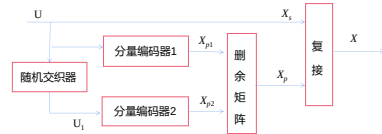


删余处理

- 一般的，Turbo码的码率只有 $1/3$ ，所以频带利用率不高，为了解决这一问题，在编码器中加入了删余处理。
- 即在编码完成后，将分量码输出的两路校验信息输入删余器，按照一定的规则删除一部分校验信息，减小信息的冗余度，从而提高编码效率。
- 一般的删余确定冗余的规则是一位信息码至多需要一个校验位来验证错误，因此，对 $1/3$ 码率的编码来说可以交替删余。
- 但Turbo码在经历交织后其第二个校验码已经不再与源码一一对应，此时交替删除还能起效的原因是因为turbo码的卷积特性使其每一位校验码对应着多位源码，因而在删余后其校验范围仍能包括整个源码。

并行级联卷积码(PCCC)

- 随机交织器：将信息序列 U 进行比特位置重排列得到 U_1 （内容不变）
- 分量编码器：生成校验序列 X_{p1} 和 X_{p2} （一般两个编码器结构相同）
- 删余矩阵：为了提高码率，周期地删除一些校验位，形成校验位序列 X_p
- 复接：未编码序列 X_u 与 X_p 经过复接后，生成Turbo码序列 X



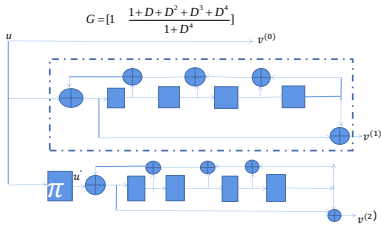
Turbo码的码率

- 若两个分量码的码率分别为 R_1 和 R_2 ，则Turbo码的码率为

$$R = \frac{R_1 R_2}{R_1 + R_2 - R_1 R_2}$$

Turbo码的编码过程举例

- 下图中，两个分量码都是(2,1,4)系统反馈编码器，具有相同的生成矩阵



注意事项

- 为了得到靠近香农限的系统性能，信息分组长度(交织器大小) K 一般比较大，通常至少几千个比特
- 对于分量码来说，一般选择相同结构，且约束长度较短，通常 $m \leq 4$
- 递归分量码会比非递归分量码有更好的性能
- 提高码率可通过删余矩阵实现
- 增加分量码和交织器会得到较低编码速率的Turbo码
- 最好的交织器能够比特以伪随机的方式进行排序，传统的块交织器(行列)在Turbo码中性能不好，除非块长度很短

Turbo码的译码

- Turbo码有一重要特点是其译码较为复杂，比常规的卷积码要复杂的多。
- 这种复杂不仅在于其译码要采用迭代的过程，而且采用的算法本身也比较复杂。
- 这些算法的关键是不但要能够对每比特进行译码，而且还要伴随着译码给出每比特译出的可靠性信息，有了这些信息，迭代才能进行下去。
- Turbo码译码算法：MAP(Maximum A Posteriori)、Max-Log-MAP、Log-MAP和SOVA(Soft Output Viterbi Algorithm)。

Turbo码的编码过程举例

- 删余矩阵的作用是提提高码率，元素取自集合{0,1}
- 矩阵的每一行对应两个分量编码器，“0”表示相应位置上的校验比特被删除，“1”表示保留相应位置的校验比特
- 对于1/3码率的Turbo编码器，输入序列 $U=(1011001)$ ，第一个分量编码器输出的校验序列 $X_{p1}=(1101000)$ ，第二个分量编码器输出的校验序列 $X_{p2}=(1001000)$ ，最终输出的序列为 $X=(111,010,100,111,000,000,100)$
- 添加删余矩阵 $P=\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 提升到1/2码率，删除第一个分量编码器偶数位校验比特，删除第二个编码器奇数位校验比特，与系统输出复接后得到的序列为 $X=(11,00,10,11,00,00,10)$

注意事项

- 由于交织器只是对比位置进行重新排序，因此，交织后的序列 u' 与原始序列 u 具有相同的重量
- 对每个分量码来说，用BCJR(或MAP)算法作为SISO译码器能够获得最好的性能；因为MAP译码器使用了前向-后向算法，信息是以块的形式进行的，因此，对第一个分量译码器来说，附加 m 个0比特能够让它返回到全0态；但对于第二个译码器来说，由于交织器的作用，将不能返回到全0态

注：BCJR算法是一种定义在网格图上的用来最大化纠错编码的后验概率的算法，主要用于卷积编码。这种算法以它的发明者的名字命名，分别是Bahl、Cocke、Jelinek和Raviv。这个算法对于迭代的纠错编码来说是非常重要的，其中包括Turbo码和LDPC码。

译码算法

- MAP算法**：MAP算法采用递归、迭代方法，将最大似然对数似函数作为软判决的输出。但由于该算法需要多次迭代，复杂度较高，运算量非常大，而且对数和指数的运算在数字电路中比较难以实现，导致其在工程实现上受到了很大程度的限制。
- Max-log-MAP算法**：即最大值运算，在MAP算法基础上通过减少格图搜索状态来达到简化的目的。Max-log-MAP对MAP所做的修改是在对数域里对一些函数进行计算，这样省去了很多的指数的运算和对数域的运算，很大程度上降低了运算的复杂度，大大简化了运算量；但是由于在计算过程有一些近似处理，所以Max-log-MAP算法不是最优的。

译码算法

- **Log-MAP算法**：即对数域算法，Robertson等人在Max-log-MAP算法基础上进行了一些修改，使得Log-MAP算法相比于Max-log-MAP算法只多了一些查表及加法运算，尽管提高了一点复杂度但是性能却得到很大的提高。
- **SOVA算法**：即软输出Viterbi译码，SOVA算法在标准Viterbi译码算法上进行了一些修改：在进行最大似然路径选择时必须参考其先验信息；不但需要给出每个比特译码的准确结果，还需要计算出每个比特译码的可靠性；通过一些计算从中获取一些关于译码比特的先验信息，从而为下次迭代做好准备，大大减小了运算量，但性能受到了损失，大约损失了1dB。

MAP译码算法

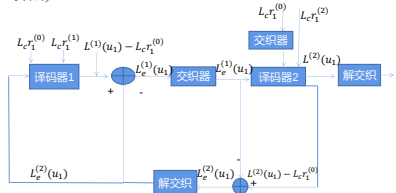
- MAP算法最初是用来估计无记忆噪声下的马尔可夫过程的，它是一种最优的算法。
- Bahl等人于1974年把它用于线性分组码和卷积码的译码中，在用于卷积码的译码时，对于给定接收序列Y，它不像Viterbi算法那样以栅格路径上的比特组错误最少为目的，而是以译码出来的符号 X_i 的错误最少为目的。
- 由于在卷积码的译码中，MAP算法要考虑栅格图中的所有可能路径，这样运算量就非常大，实际系统中很少用到。

MAP译码算法

- 虽然MAP算法早在1974年就被提出，但一直未被得到充分利用。
- 1993年Turbo码被提出来，MAP算法被用于Turbo码的译码之后，这种算法才得到广泛的应用。
- MAP算法不仅能译出序列的比特值，在译码的同时还能输出关于每比特译出的可靠性信息。这种特点正好符合了Turbo码的迭代译码特性，因此，被用于Turbo码的译码中。

迭代译码器

- 假设速率 $R = 1/3$ ，即没有删余处理的并行级联码，它使用了两个SISO译码器(MAP 算法)



MAP译码算法

- 每个时间单元，从信道接收到三个输出值，一个是对应着信息比特 u_1 ， $v_1^{(0)}$ ，用 $r_1^{(0)}$ 表示，其他两个对应着校验比特 $v_1^{(1)}$ 和 $v_1^{(2)}$ ，分别用 $r_1^{(1)}$ 和 $r_1^{(2)}$ 表示，这样接收到的向量(3K维)为 $r = (r_0^{(0)} r_0^{(1)} r_0^{(2)}, r_1^{(0)} r_1^{(1)} r_1^{(2)}, \dots, r_{K-1}^{(0)} r_{K-1}^{(1)} r_{K-1}^{(2)})$
- 现在假设每个发送比特的映射关系为 $0 \rightarrow -1$ 和 $1 \rightarrow +1$ ，软输出的AWGN 信道。

MAP译码算法

- 给定接收值 $r_1^{(0)}$ ，发送信息比特 u_1 的对数似然值(L值)定义为

$$L(v_1^{(0)} | r_1^{(0)}) = L(u_1 | r_1^{(0)}) \text{ (译码前):}$$

$$\begin{aligned} L(u_1 | r_1^{(0)}) &= \ln \frac{p(u_1 = +1 | r_1^{(0)})}{p(u_1 = -1 | r_1^{(0)})} = \ln \frac{p(r_1^{(0)} | u_1 = +1) p(u_1 = +1)}{p(r_1^{(0)} | u_1 = -1) p(u_1 = -1)} = \ln \frac{p(r_1^{(0)} | u_1 = +1)}{p(r_1^{(0)} | u_1 = -1)} + \ln \frac{p(u_1 = +1)}{p(u_1 = -1)} \\ &= \ln \frac{e^{-(E_s/N_0)(r_1^{(0)} - u_1)^2}}{e^{-(E_s/N_0)(r_1^{(0)} + u_1)^2}} + \ln \frac{p(u_1 = +1)}{p(u_1 = -1)} = 4 \frac{E_s}{N_0} r_1^{(0)} u_1 + \ln \frac{p(u_1 = +1)}{p(u_1 = -1)} = L_c r_1^{(0)} + L_a(u_1) \end{aligned}$$

其中 $\frac{E_s}{N_0}$ 是信道SNR， u_1 和 $r_1^{(0)}$ 都是用 $\sqrt{E_s}$ 归一化的， $L_c = 4E_s/N_0$ 是信道可靠度因子， $L_a(u_1)$ 是比特 u_1 的先验L值。

MAP译码算法

- 在发送校验比特是 $v_1^{(j)}$ 的情况下，给定接收到的值为 $r_1^{(j)}$, $j=1,2$, L 值(译码前)为：

$$L(v_1^{(j)}|r_1^{(j)})=L_c r_1^{(j)} + L_a(v_1^{(j)}) = L_c r_1^{(j)}, j = 1, 2$$

- 因为在一个线性码中信息比特等概率，校验比特为+1和-1的概率也是相同的，所以校验比特的先验 L 值为0，即

$$L_a(v_1^{(j)})=\ln \frac{p(v_1^{(j)}=+1)}{p(v_1^{(j)}=-1)}=0, \quad j=1, 2$$

- 对于译码器1的第一次迭代， $L_a(u_1)$ 也等于0，之后，信息比特的先验 L 值就被另一个译码器输出的外部 L 值所取代。

MAP译码算法

- 接收到的信道 L 值(软信息) $L_c r_1^{(0)}$ (for u_1)和 $L_c r_1^{(1)}$ (for $v_1^{(1)}$)进入译码器1，交织后的信道 L 值(软信息) $L_c r_1^{(0)}$ (for u_1) 和 $L_c r_1^{(2)}$ (for $v_1^{(2)}$) 进入译码器2，译码器1的输出包括两项：

- 给定接收到的向量(部分) $r_1 \triangleq [r_0^{(0)}, r_0^{(1)}, r_1^{(0)}, r_1^{(1)}, \dots, r_{K-1}^{(0)}, r_{K-1}^{(1)}]$ 以及先验输入向量 $L_a^{(1)} \triangleq [L_a^{(1)}(u_0), L_a^{(1)}(u_1), \dots, L_a^{(1)}(u_{K-1})]$ ，每个信息比特的后验 L 值(译码后)为

$$L^{(1)}(u_1) = \ln \left[\frac{p(u_1 = +1|r_1, L_a^{(1)})}{p(u_1 = -1|r_1, L_a^{(1)})} \right]$$

MAP译码算法

- 与每个信息比特相关的外部后验概率值(译码后)

$$(u_1) = L^{(1)}(u_1) - [L_c r_1^{(0)} + L_e^{(2)}(u_1)]$$

该信息经交织后送给译码器2，作为先验值 $L_a^{(2)}(u_1)$

- 同样地，译码器2的输出也包括两项：

- $L^{(2)}(u_1) = \ln \left[\frac{p(u_1 = +1|r_2, L_a^{(2)})}{p(u_1 = -1|r_2, L_a^{(2)})} \right]$ ，其中 r_2 是(部分)接收向量， $L_a^{(2)}$ 是先验输入向量。

MAP译码算法

- $L_e^{(2)}(u_1) = L^{(2)}(u_1) - [L_c r_1^{(0)} + L_e^{(1)}(u_1)]$ 是译码器2产生的外部后验 L 值，经过解交织后，作为先验值 $L_a^{(1)}(u_1)$ 反馈作为译码器1的输入。这样，每个译码器的输入包含三项：软信道 L 值 $L_c r_1^{(0)}$ 、 $L_c r_1^{(1)}$ (或 $L_c r_1^{(2)}$)，以及从另一个译码器传送的外部后验 L 值 $L_e^{(2)}(u_1)=L^{(1)}(u_1)$ (或 $L_e^{(1)}(u_1)$)
 $L_a^{(2)}(u_1)$ 。

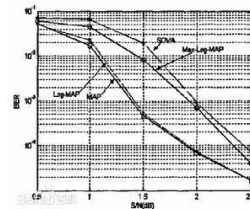
注意：在译码器1的初始迭代时，外部后验 L 值 $L_e^{(2)}(u_1)$

$L_a^{(1)}(u_1)$ 就是初始先验 L 值 $L_a(u_1)$ (对于等概信息比特，该值为0)，在译码器1的随后迭代中，先验 L 值 $L_a^{(1)}(u_1)$ 就用接收到的外部后验 L 值 $L_e^{(2)}(u_1)$ (进行解交织处理后)代替。

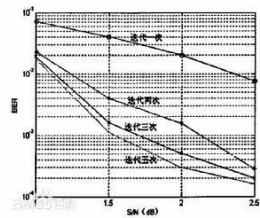
MAP译码算法

- 对于译码器2，其第一次迭代和随后的迭代都是相同的，外部后验 L 值一直都是 $L_e^{(1)}(u_1)$ 。译码迭代处理，每个译码器将其外部 L 值传送给另一个译码器，形成一个Turbo的效果，使得译码结果越来越可靠。
- 经过一定次数的迭代后，译码后的信息比特就从译码器2输出的后验 L 值 $L^{(2)}(u_1)$ 中判决得到，如果该 L 值为正，就判为+1，否则就判为-1。
- 值得一提的是：Turbo的很多特性与LDPC很类似，如编码方案都是产生随机重量分布的码、译码方法都是在迭代过程中利用了后验概率似然值、都利用了外部信息的概念等。实际上，正是由于Turbo码的出现，才使得人们对LDPC码的优点有了重新的认识。

不同译码算法对Turbo码性能的影响



不同迭代次数对Turbo码性能的影响



未来展望

- **亟待解决的问题**：译码算法的改进、复杂性的降低、译码时延的减小
- **研究方向**：最优分量码与交织器的联合设计、低复杂性译码算法、译码迭代过程的优化、收敛性以及迭代停止准则的设计、联合信道估计/多用户检测/均衡和译码算法、Turbo码与高阶调制技术的结合、Turbo编译码器的硬件实现、Turbo码在无线通信、移动通信以及多媒体通信中的应用